

SymPy Bug Report

1 Bug 29: solveset returns a negative solution for a principal fractional power

1.1 Minimal Reproducer

```
from sympy import Eq, N, Rational, S, simplify, solveset, symbols

x = symbols("x")
sol = solveset(Eq(x**Rational(2, 3), 4), x, S.Reals)
print("solution =", sol)
for s in sol:
    residual = (x**Rational(2, 3) - 4).subs(x, s)
    print("residual at", s, "=", simplify(residual))
    print("numeric residual at", s, "=", N(residual, 50))
```

1.2 Observed Behavior

```
solution = {-8, 8}
residual at -8 = -4 + 4*(-1)**(2/3)
numeric residual at -8 = -6.0 + 3.4641016151377545870548926830117447338856105076208*I
residual at 8 = 0
numeric residual at 8 = 0
```

SymPy 1.14.0 returns both -8 and 8 from the real-domain call. The positive value satisfies the original equation, but the negative value leaves a nonzero complex residual when substituted into SymPy's principal-power expression.

1.3 Expected Behavior

The real-domain solution set should be $\{8\}$. A value returned by `solveset(..., S.Reals)` should satisfy the original equation under the expression semantics used by SymPy, and -8 does not satisfy $x^{2/3} = 4$ for SymPy principal powers.

1.4 Mathematical Explanation

The equation is

$$x^{2/3} = 4.$$

For positive real x , this behaves as expected: $8^{2/3} = 4$. For a negative base, however, SymPy's `Pow` uses the principal complex branch for non-integer rational exponents rather than the real cube-root convention. Therefore

$$(-8)^{2/3} = 4(-1)^{2/3} = -2 + 2\sqrt{3}i,$$

so

$$(-8)^{2/3} - 4 = -6 + 2\sqrt{3}i \neq 0.$$

Thus the extra value is not an equivalent representation, a harmless branch choice, or a missing simplification. It is a false solution to the equation actually represented by the input expression.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant logic in `sympy/solvers/solveset.py`. The call `solveset( $x^{2/3} = 4$ ,  $x$ ,  $\mathbb{R}$ )` is normalized to solving $x^{2/3} - 4 = 0$. In `_solveset`, the expression is treated as a polynomial in the fractional-power generator $x^{2/3}$. After solving for that generator, the code around `sympy/solvers/solveset.py:1057–1062` calls the real inversion helper to invert $y = x^{2/3}$ back to values of x .

The inversion then reaches `_invert_real` in `sympy/solvers/solveset.py`. The reported faulty branch is the rational-power branch around lines 284–289:

```
if den % 2 == 1:
    root = Lambda(n, real_root(n, expo))
    res = imageset(root, g_ys)
    if num % 2 == 0:
        neg_res = imageset(Lambda(n, -n), res)
        return _invert_real(base, res + neg_res, symbol)
```

For exponent $2/3$, the denominator is odd and the numerator is even. This rule therefore returns both inverse branches, $y^{3/2}$ and $-y^{3/2}$. That is appropriate for the real-valued expression $(\text{real.cuberoot}(x))^2$, but it is not the inverse of SymPy's principal complex power $x^{2/3}$. With $y = 4$, this produces both 8 and -8 . The diagnosis also notes that no later residual check filters the invalid negative branch against the original principal-power expression. A plausible fix direction is to distinguish principal `Pow` semantics from explicit `real_root` semantics, filtering or guarding the negative branch for principal powers with even numerator and odd denominator.

1.6 Additional Failing Instances

The same inversion error applies to the family

$$\text{solveset}(x^{2/3} = m^2, x, \mathbb{R})$$

for positive integers m . The expected solution is always $x = m^3$, because $(m^3)^{2/3} = m^2$, while the negative branch $x = -m^3$ is a solution only under real-root semantics, not under SymPy's principal `Pow` semantics. The verification artifact parametrizes the representative case with $m = 2, 3, 4, 5, 6, 7$.

Input / Expression	SymPy behavior	Expected behavior
$x^{2/3} = 2^2, x \in \mathbb{R}$	returns $\{-8, 8\}$	$\{8\}$
$x^{2/3} = 3^2, x \in \mathbb{R}$	returns $\{-27, 27\}$	$\{27\}$
$x^{2/3} = 4^2, x \in \mathbb{R}$	returns $\{-64, 64\}$	$\{64\}$
$x^{2/3} = 5^2, x \in \mathbb{R}$	returns $\{-125, 125\}$	$\{125\}$
$x^{2/3} = 6^2, x \in \mathbb{R}$	returns $\{-216, 216\}$	$\{216\}$
$x^{2/3} = 7^2, x \in \mathbb{R}$	returns $\{-343, 343\}$	$\{343\}$

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest related public material concerns the `solveset` interface, SymPy’s branch-sensitive elementary functions, and the general mathematical fact that negative bases with non-integer rational exponents are branch-sensitive. The artifact found no public issue, pull request, Stack Overflow question, mailing-list thread, or release note describing this exact reproducer, returned set $\{-8, 8\}$, and failing residual at -8 . The deduplication verdict is therefore a new instance of a known failure mode: the broader principal-branch issue is recognizable, but this exact real-domain `solveset` counterexample is previously unreported in the available evidence and remains individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import Eq, FiniteSet, Integer, Rational, S, solveset, symbols

@pytest.mark.parametrize("m", [2, 3, 4, 5, 6, 7])
def test_solveset_principal_two_thirds_power_excludes_negative_branch(m):
    x = symbols("x")
    sol = solveset(Eq(x**Rational(2, 3), Integer(m)**2), x, S.Reals)
    assert sol == FiniteSet(Integer(m)**3)
```